pfi

Python for Fixed Income

COURSE 9 · DEPLOYMENT

What the Server Installs

WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: what the server installs

Today

4.1 What the course environment holds

4.2 What a file needs: requirements_for

4.3 What a list costs: installed_size, Windows and Linux

4.4 The limits, as read today

4.5 The file the server reads: pins_check, not pip freeze

4.6 Four Pythons

4.7 The desk's decision

The question

What does this code import, and what does that cost to install?

A server has none of your laptop: it installs its requirements file, on a fresh Linux machine, every build.

Offline today: every size is read from files Python already has on disk.

The rule for servers

A server installs only what its code imports.

Vercel	one bundle, built from requirements.txt
A Space	Gradio preinstalled; the rest from requirements.txt
GitHub Actions	a fresh machine: installs on every run

On a platform's figures

Allowed names

included in Hobby as read

the free tier as read

the platform line

Never called

guaranteed

unlimited

free forever

What the course environment holds

199

distributions installed, 2026-10-13

3,558,784,938

bytes on disk, by RECORD

3.31 GB

1024^3 bytes, Windows

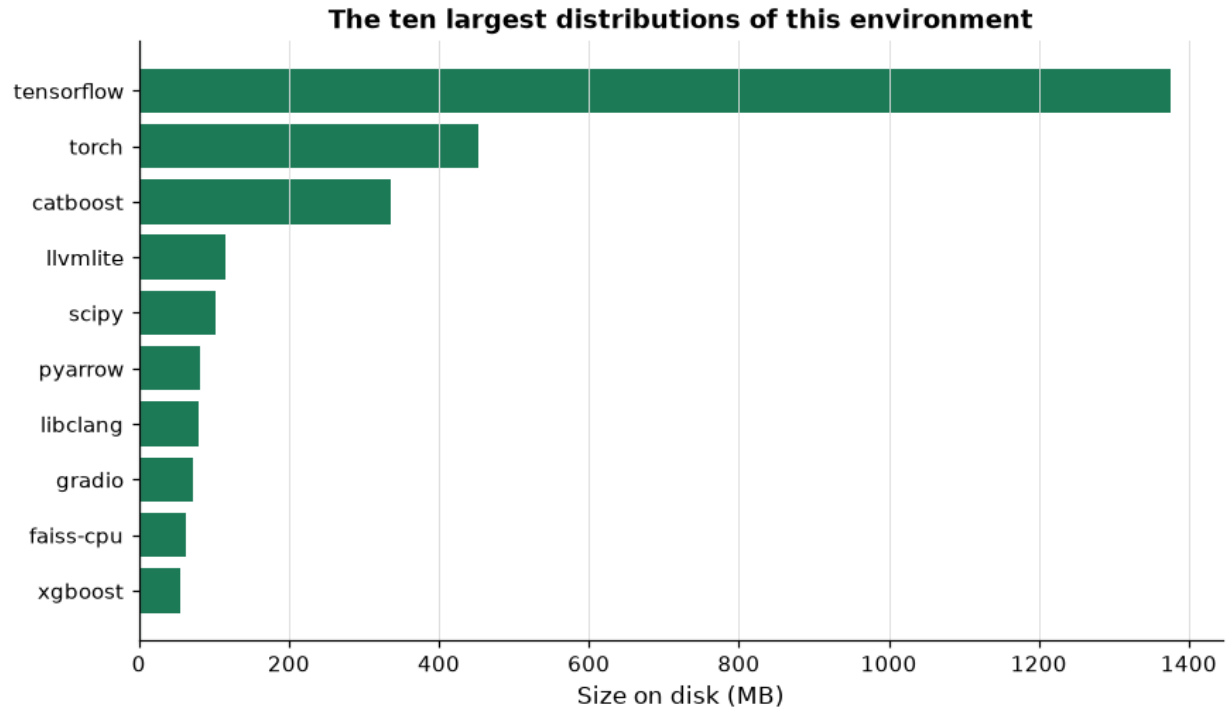
RECORD

Every distribution lists each file it installed, with its size in bytes. Add them up.

```
sum(f.size or 0 for f in d.files or [])
```

None of it goes to a server: shipping your environment sends TensorFlow to answer a DV01 request.

The largest distributions



Measured 2026-10-13

199 distributions

3.31 GB on disk

A few distributions hold most of the bytes. A server that does not import them does not install them.

How requirements_for works

The problem	A server installs what the code imports, so the list can be read from the code
The method	ast reads each import line without running anything; keep the top-level name
Then	Drop the standard library and local modules; map each name to its distribution and pin it
You control	The files, and which functions the server will really run
What comes out	Requirement lines: an upper bound, never a measurement of what runs

What a file needs: requirements_for

Import name, distribution

`import sklearn`	`scikit-learn`
`import yaml`	`PyYAML`
`import dotenv`	`python-dotenv`
`from PIL import`	`pillow`

Three servers

bondmath.py	nothing: the standard library
the bond service	`fastapi==0.143.0`
the desk, seven files	20 distributions

An upper bound: it counts imports inside functions a server may never run. Read it, keep what runs.

How installed_size works

The problem	One requirements line installs more than one distribution
The method	Follow each distribution's Requires-Dist, each once, on this machine
Each step	Leave out extras, evaluate the marker here, add up the RECORD sizes
You control	The starting lines
What comes out	One row for each distribution, sized on this machine, not on the server's Linux

What a list costs, Windows and Linux

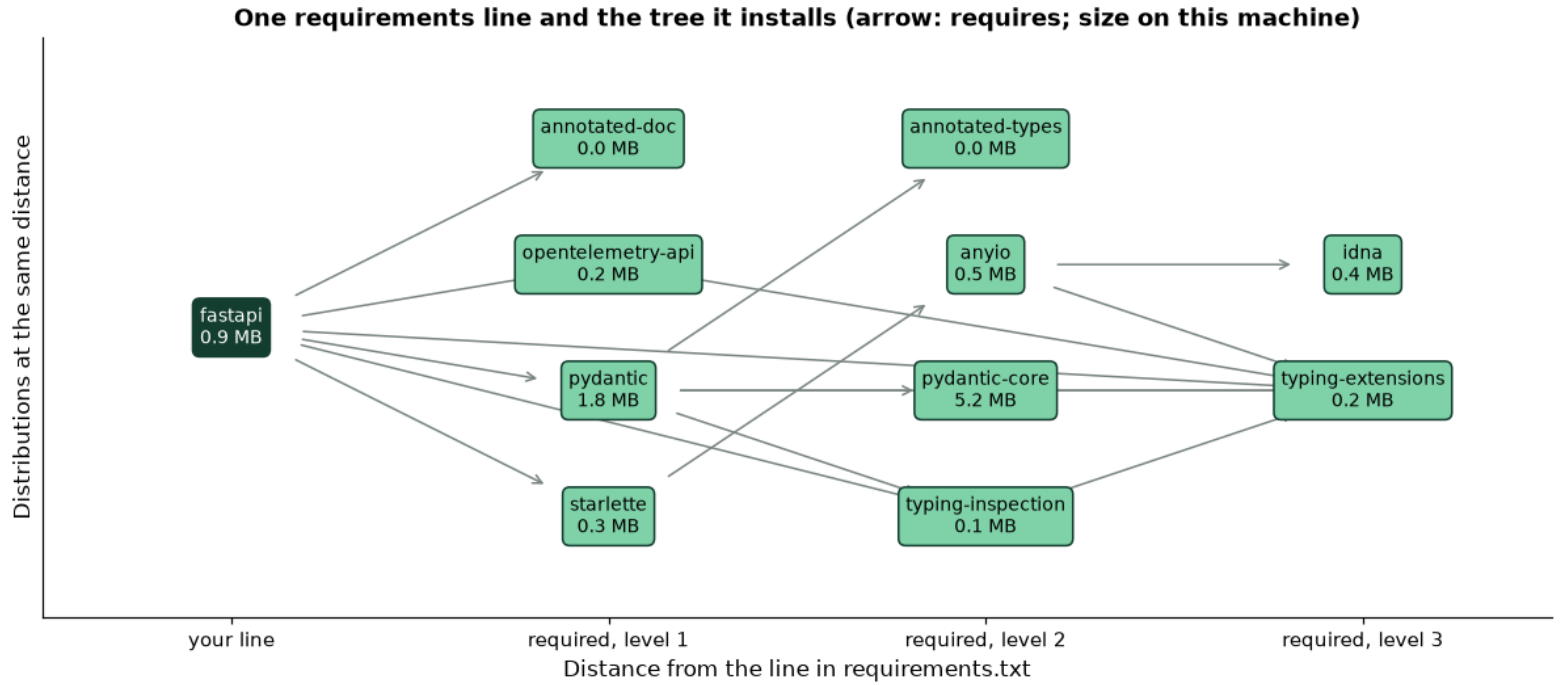
Server	Requirements	Windows	Linux
the bond service	1 line	11 distributions, 9.6 MB	9.2 MB
the desk	20 lines	94 distributions, 923.4 MB	109 wheels, 5,716.5 MB

MB of 1024 x 1024 bytes. Linux: the course build's measurement of 2026-10-12, wheels downloaded, never installed.

On Linux, torch from PyPI is the CUDA build: NVIDIA's GPU libraries come as separate wheels.

A wheel is built for one platform: measure a server's size on the server's platform.

One line, and the tree it installs



How to read it

Dark box

the line you write

Arrow

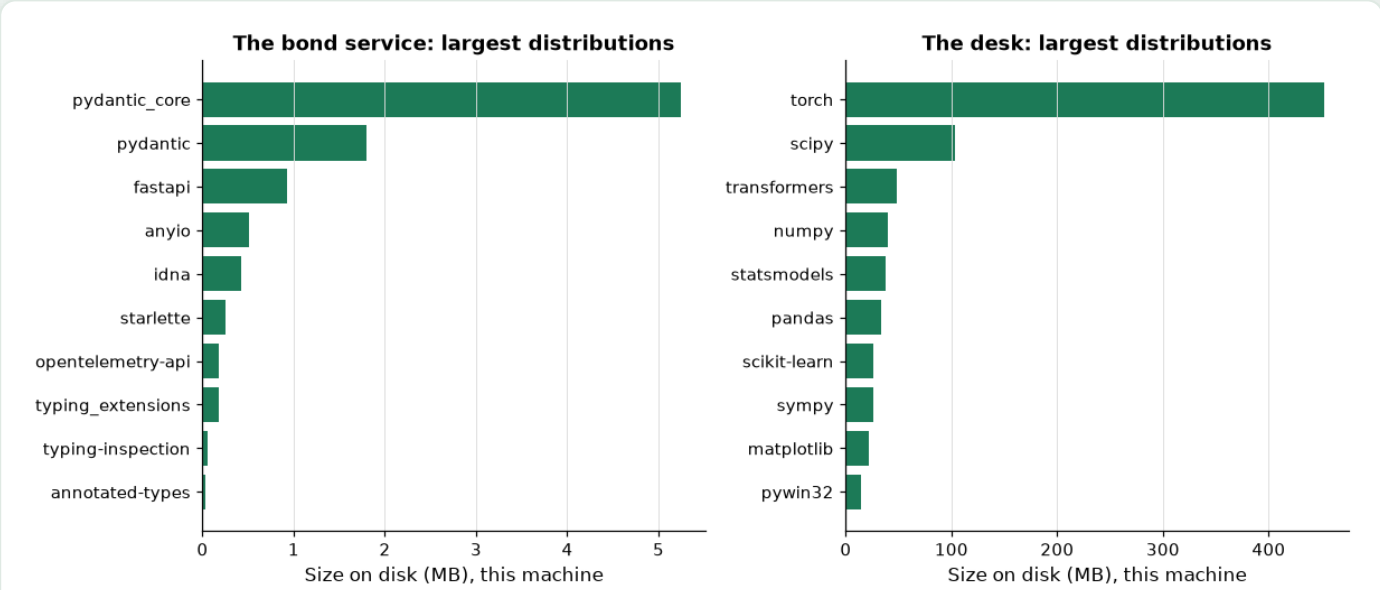
requires

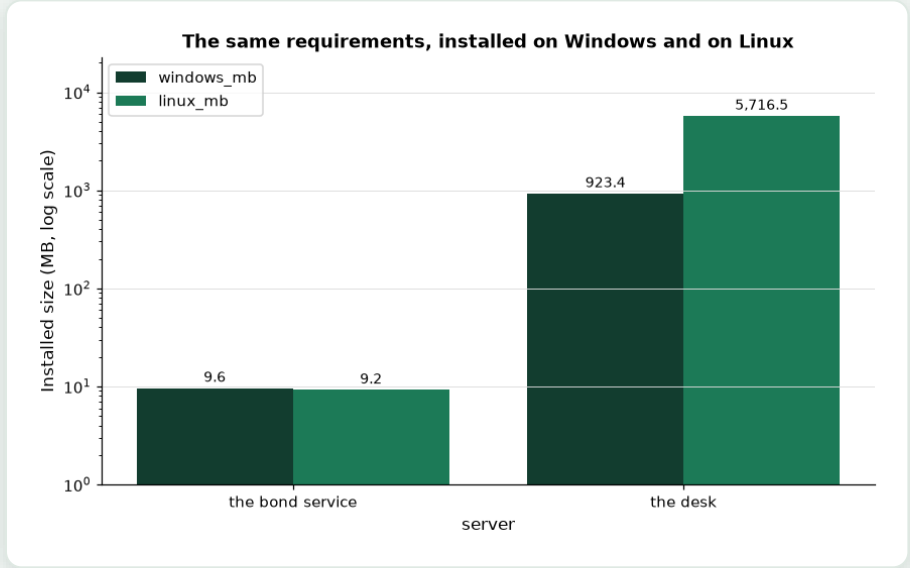
Many arrows in

installed once

One line, 11 distributions in 4 levels, 16 requirements between them. You write one line; pip follows every arrow.

The two lists, drawn





The bond service: 9.6 MB on Windows, 9.2 on Linux. The desk: 923.4 MB and 5,716.5. A wheel is built for one platform.

The limits, as read today

Vercel

500 MB uncompressed bundle

A Space

Gradio from sdk_version

GitHub Actions

installs on every run

Vercel Python: a 500 MB uncompressed bundle limit (vercel.com/docs/functions/runtimes/python, read 2026-10-12).

Spaces: Gradio from sdk_version, the rest from requirements.txt (huggingface.co/docs/hub/spaces-dependencies, read 2026-10-12).

GitHub Actions: free on public repositories (docs.github.com/en/billing/concepts/product-billing/github-actions, read 2026-10-12).

The bond service fits with room to spare; the desk on Linux does not come close.

How a requirements file is read

The problem	A file that reads one way today can install another way on the build day
A line	A name; optional extras, a version rule, and after a semicolon a marker; # starts a comment
The rules	== pins one version; a marker is read on the machine that runs pip
pins_check	Pinned, a marker on Windows-only names, no name twice
What comes out	A sentence for each problem; an empty list means the same file on every build

pins_check, and why not pip freeze

unpinned

the newest, whenever it builds

Windows-only

pywin32 with no marker

listed twice

FastAPI and fastapi

mcp's own requirement: a marker

```
["pywin32>=311; sys_platform == 'win32'"]
```

pip freeze: 198 lines, all pinned; pins_check:

```
line 140: pywin32 installs only on Windows; add ; sys_platform == "win32"
```

Write a server's file from the code, read it, check it. Never pip freeze.

A requirements file, line by line

Line	As written	pip on Linux	pins_check
1	`fastapi==0.143.0`	installs it	ok
2	`uvicorn>=0.50  # a range, not a pin`	installs it	uvicorn>=0.50 is not pinned with ==
3	`pywin32==312`	installs it	pywin32 installs only on Windows; add ; sys_platf...
4	`# only a comment`	skips it	skipped: a comment
5	`pyyaml==6.0.3`	installs it	ok
6	`PyYAML==6.0.3`	installs it	pyyaml is listed twice
7	`colorama==0.4.6; sys_platform == "win...`	skips it	ok

pip reads one line at a time. pins_check says what no pip run says until it fails.

Four Pythons

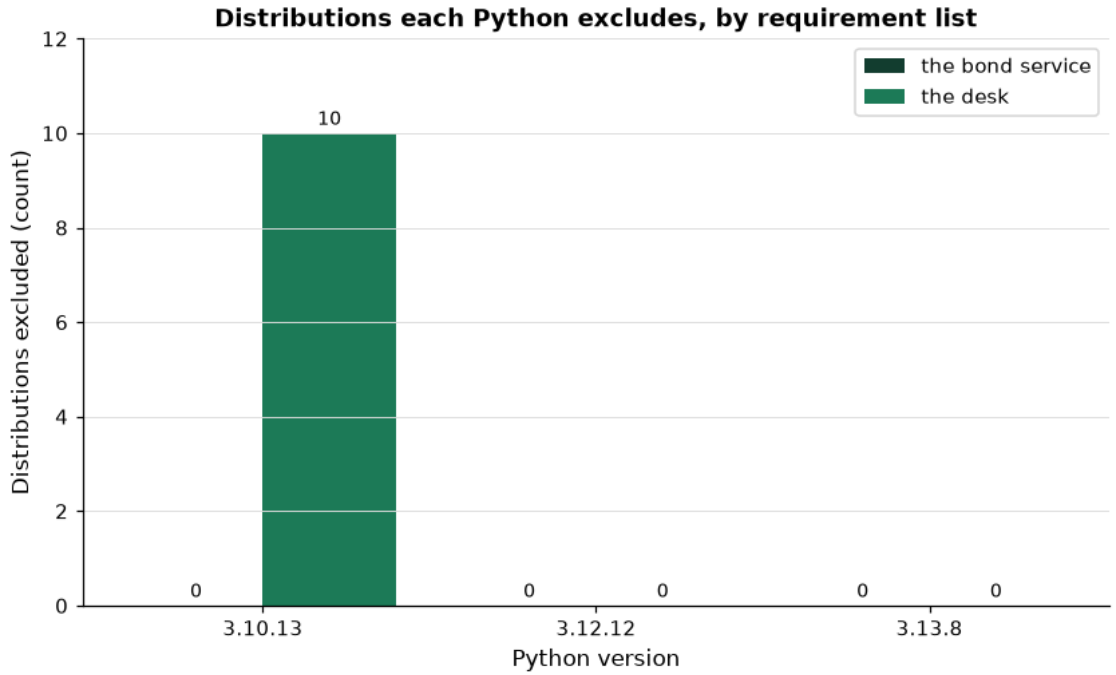
Where	Python	Set by
your laptop	3.13.8	the course's .venv
GitHub Actions	3.12 and 3.13	the workflow's matrix
Vercel	3.13	.python-version
a ZeroGPU Space	3.12.12	python_version in the README

```
Python 3.10.13: the bond service 0 excluded; the desk 10 excluded
Python 3.12.12: the bond service 0 excluded; the desk 0 excluded
```

Spaces: python_version defaults to 3.10 (huggingface.co/docs/hub/spaces-config-reference, read 2026-10-12).

State the Python everywhere, quoted in YAML.

Which Python excludes what



Python	Bond service	Desk
3.10.13	0	10
3.12.12	0	0
3.13.8	0	0

Only the desk on 3.10 has bars: a Space left at its default would fail on its first build.

The desk's decision

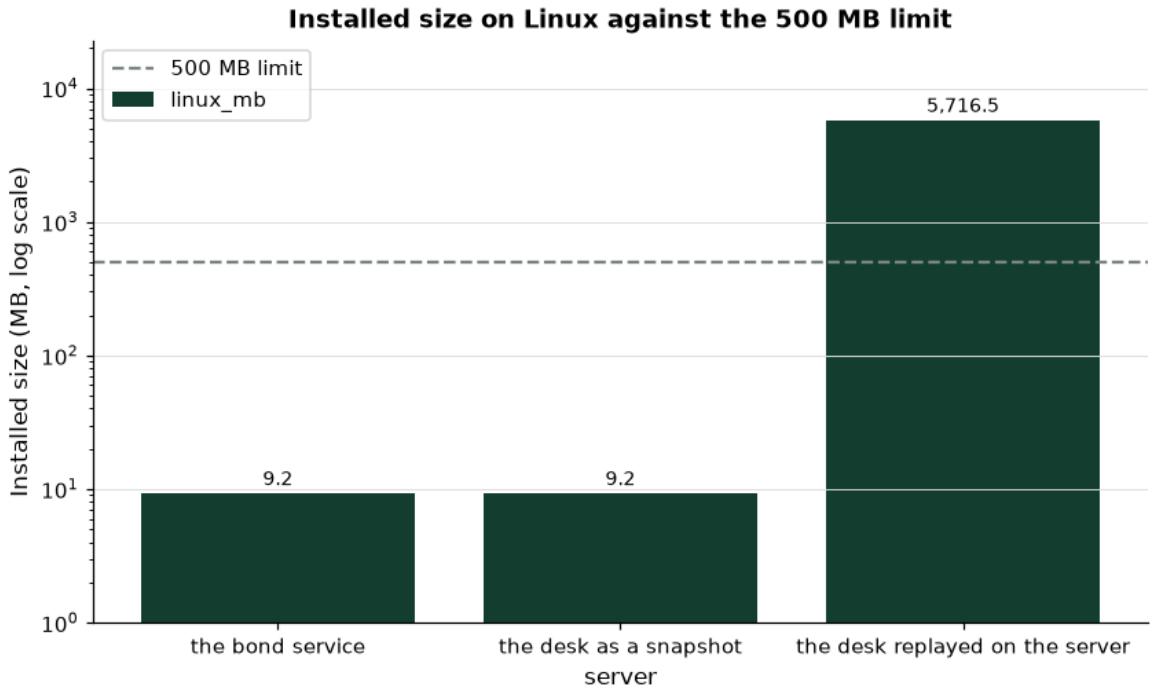
```
                        server  requirements  linux_mb against 500 MB
               the bond service             1       9.2           fits
          the desk as a snapshot             1       9.2           fits
  the desk replayed on the server            20    5716.5 over, 11 times
```

Vercel Python: a 500 MB uncompressed bundle limit (vercel.com/docs/functions/runtimes/python, read 2026-10-12).

Week 2 serves a snapshot: the brief as of 2026-10-02, every figure traced to a saved exchange by its hash.

Where it fails: requirements_for reads imports, not behaviour. A file loaded at run time or a plugin found by name is not an import. The first build is the real test.

Against the limit



The decision

the snapshot ships

the replay stays on your laptop

Two bars sit well under the dashed line and one stands above it.

Excel and Git

```
E1:  =SUM(C2:C200)/1024^2
E2:  =IF(E1>500,"over","fits")
```

the bond service: 9.6 MB, fits

the desk: 923.4 MB, over

Excel and pandas: sizes_service.csv: 11 rows, 9.6 MB, fits | sizes_desk.csv: 94 rows, 923.4 MB, over

```
requirements-server.txt: only what the server runs
requirements-server.txt for the bond service fold...
deploykit day 4: requirements_for, installed_size...
Start of day 4: deploykit through day 3, 14 tests
```

```
 fastapi==0.143.0
-packaging==26.3
-pandas==3.0.6
-PyYAML==6.0.3
```

18 passed. requirements-server.txt keeps only what the server runs: one line.

The AI check

The docstring is the prompt. The deploy code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the lines name the import (sklearn, yaml, dotenv, PIL), not the distribution pip installs.

Code that runs is not the same as code that is right.

What you can do now

- Measure an environment from RECORD
- requirements_for: an upper bound, read it
- installed_size: Windows is not Linux
- pins_check, never pip freeze
- Four Pythons; the desk ships as a snapshot

Week 1, Day 5 Case study: bondmath through CI